

Clase completa de preparación

Evaluación Final Transversal

FPY1101 - Fundamentos de Programación

Caso FlorExpress

Tipo de documento: Guía de clase con solución comentada
Lenguaje: Python
Enfoque: Nivel inicial, paso a paso
Duración sugerida: 90 a 120 minutos

Índice

1. Introducción de la clase	2
2. Objetivos de aprendizaje	2
3. Mejoras incorporadas al instrumento	3
4. Rúbrica sugerida de 100 puntos	3
5. Restricciones del desarrollo	3
6. Aclaración sobre los puntos suspensivos	4
6.1. Ejemplo correcto	4
6.2. Ejemplo incorrecto para esta evaluación	4
7. Uso de diccionarios globales	4
8. Cambio de tamaño por tamaño	4
9. Mensajes de error definidos	5
10 Estructura general del sistema	5
11 Explicación de los diccionarios	5
11.1 Diccionario arreglos	5
11.2 Diccionario bodega	6
12 Código completo funcional y comentado	6
13 Pruebas rápidas sugeridas	13
13.1 Prueba 1: buscar unidades por tipo	13
13.2 Prueba 2: búsqueda por rango de precio	13
13.3 Prueba 3: precio incorrecto	13
13.4 Prueba 4: agregar arreglo válido	14
13.5 Prueba 5: eliminar arreglo existente	14
14 Errores frecuentes que deben evitarse	14
14.1 Copiar los puntos suspensivos	14
14.2 No convertir el código a mayúscula	14
14.3 No validar números	14
15 Guion sugerido para desarrollar la clase	15
16 Cierre de la clase	15

1. Introducción de la clase

Estimados estudiantes:

En esta clase revisaremos una propuesta completa para resolver un sistema de administración de arreglos florales para la florería **FlorExpress**. El objetivo no es memorizar una solución, sino comprender cómo organizar un programa completo utilizando los contenidos esenciales de **Fundamentos de Programación**.

Durante la actividad trabajaremos con:

- variables;
- condicionales;
- ciclos;
- listas;
- diccionarios;
- funciones;
- validaciones de entrada;
- manejo de excepciones con `try/except`;
- menú interactivo.

El sistema debe permitir consultar, buscar, actualizar, agregar y eliminar arreglos florales. Para ello se usarán dos diccionarios relacionados por el mismo código de arreglo.

Idea central: un buen programa no solo debe funcionar. También debe estar ordenado, ser fácil de leer, validar los datos ingresados y evitar errores cuando el usuario escribe información incorrecta.

2. Objetivos de aprendizaje

Al finalizar esta clase, el estudiante debería ser capaz de:

1. Crear y manipular diccionarios en Python.
2. Relacionar dos diccionarios mediante una misma clave.
3. Crear funciones simples y reutilizables.
4. Validar datos ingresados por teclado.
5. Aplicar manejo de excepciones para evitar errores de ejecución.
6. Construir un menú interactivo.
7. Implementar operaciones básicas de consulta, búsqueda, actualización, agregado y eliminación.

3. Mejoras incorporadas al instrumento

Para fortalecer la claridad del examen, se incorporan las siguientes mejoras:

1. Agregar una rúbrica de 100 puntos.
2. Cambiar tamaño por tamaño en firmas de funciones.
3. Aclarar la validación de rangos de precio.
4. Aclarar la validación del nuevo precio.
5. Indicar explícitamente que los diccionarios serán globales.
6. Aclarar que los puntos suspensivos ... no se copian en el código.
7. Agregar restricciones sobre herramientas permitidas.
8. Definir mensajes de error para los casos importantes.

4. Rúbrica sugerida de 100 puntos

Criterio de evaluación	Puntaje
Definición correcta de los diccionarios iniciales arreglos y bodega.	10 pts
Implementación del menú principal y ciclo de ejecución.	10 pts
Función unidades_tipo(tipo).	10 pts
Función busqueda_precio(p_min, p_max).	15 pts
Función actualizar_precio(codigo, nuevo_precio).	10 pts
Función agregar_arreglo(...).	15 pts
Funciones de validación independientes.	15 pts
Función eliminar_arreglo(codigo).	10 pts
Manejo de excepciones, mensajes claros y salida limpia.	5 pts
Total	100 pts

5. Restricciones del desarrollo

Para mantener la evaluación dentro del nivel de la asignatura, la solución debe respetar estas restricciones:

- No usar bases de datos.
- No usar archivos externos.
- No usar librerías externas.
- No usar interfaces gráficas.
- No usar programación orientada a objetos.
- Resolver únicamente con variables, listas, diccionarios, condicionales, ciclos, funciones y try/except.

6. Aclaración sobre los puntos suspensivos

En algunos ejemplos de enunciados se usa:

```
1 ...
```

Eso significa que pueden existir más registros, pero **no debe copiarse literalmente dentro del código final**.

6.1. Ejemplo correcto

```
1 arreglos = {  
2     "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"],  
3     "FL02": ["Caja Elegante", "caja", "blanco", "L", True, "todo año"]  
4 }
```

6.2. Ejemplo incorrecto para esta evaluación

```
1 arreglos = {  
2     "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"],  
3     ...  
4 }
```

7. Uso de diccionarios globales

En esta solución, los diccionarios arreglos y bodega se definen al inicio del programa y estarán disponibles durante toda la ejecución.

Esto permite que las funciones trabajen con ellos directamente, manteniendo un nivel adecuado para estudiantes de primer curso.

Nota técnica: en cursos posteriores se podría mejorar el diseño enviando los diccionarios como parámetros o usando programación orientada a objetos. Sin embargo, para Fundamentos de Programación, el uso de diccionarios globales simplifica la comprensión inicial.

8. Cambio de tamaño por tamaño

Aunque Python 3 permite usar caracteres como la letra ñ, se recomienda evitarla en nombres de variables y funciones para disminuir problemas de teclado, codificación o copia de código.

Por eso, usaremos:

```
1 tamaño
```

en lugar de:

```
1 tamaño
```

La firma recomendada para agregar un arreglo será:

```
1 agregar_arreglo(codigo, nombre, tipo, color_principal, tamaño,  
2                 incluye_tarjeta, temporada, precio, unidades)
```

9. Mensajes de error definidos

Situación	Mensaje
Opción del menú inválida	Debe seleccionar una opcion valida
Valor numérico no entero	Debe ingresar valores enteros
Precio menor que cero en búsqueda	Los precios deben ser mayores o iguales a cero
Precio mínimo mayor que máximo	El precio minimo no puede ser mayor que el precio maximo
Precio nuevo menor o igual a cero	El precio debe ser mayor que cero
Código vacío	El codigo no puede estar vacio
Código existente al agregar	El codigo ya existe
Código inexistente	El codigo no existe
Texto vacío	El campo no puede estar vacio
Tamaño inválido	El tamano debe ser S, M o L
Respuesta de tarjeta inválida	Debe ingresar s o n
Unidades negativas	Las unidades deben ser mayores o iguales a cero
Sin resultados en búsqueda	No hay arreglos en ese rango de precios.
Salida del programa	Programa finalizado.

10. Estructura general del sistema

El programa tendrá un menú principal con seis opciones:

```

1 ===== MENU PRINCIPAL =====
2 1. Unidades por tipo de arreglo
3 2. Búsqueda de arreglos por rango de precio
4 3. Actualizar precio de arreglo
5 4. Agregar arreglo
6 5. Eliminar arreglo
7 6. Salir
8 =====

```

La ejecución se repite hasta que el usuario seleccione la opción 6.

11. Explicación de los diccionarios

11.1. Diccionario arreglos

El diccionario arreglos guarda información descriptiva:

```

1 "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"]

```

Posición	Dato
0	Nombre
1	Tipo
2	Color principal
3	Tamaño
4	Incluye tarjeta
5	Temporada

11.2. Diccionario bodega

El diccionario bodega guarda información operativa:

```
1 "FL01": [15990, 8]
```

Posición	Dato
0	Precio
1	Unidades disponibles

12. Código completo funcional y comentado

El siguiente código puede ejecutarse directamente en Python. Está comentado con intención pedagógica para que el estudiante comprenda qué hace cada parte.

```
1 # =====
2 # Evaluacion Final Transversal - Fundamentos de Programacion
3 # Caso: FlorExpress
4 # =====
5 # Este programa permite administrar arreglos florales usando:
6 # - Diccionarios
7 # - Listas
8 # - Funciones
9 # - Condicionales
10 # - Ciclos
11 # - Validaciones
12 # - Manejo de excepciones
13 # =====
14
15 # =====
16 # 1. DICCIONARIOS GLOBALES
17 # =====
18 # Estos diccionarios existen desde que inicia el programa
19 # y permanecen disponibles durante toda la ejecucion.
20 #
21 # Ambos diccionarios usan el mismo codigo como clave.
22 # Por ejemplo: "FL01" existe en arreglos y tambien en bodega.
23 #
24 # Nota docente:
25 # Se usan variables globales para mantener el nivel adecuado de FPY1101.
26 # En cursos posteriores, una version mas avanzada podria enviar los
27 # diccionarios como parametros o usar programacion orientada a objetos.
28 # =====
29
30 arreglos = {
31     "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"],
32     "FL02": ["Caja Elegante", "caja", "blanco", "L", True, "todo año"],
33     "FL03": ["Ramo Solar", "ramo", "amarillo", "S", False, "verano"],
34     "FL04": ["Centro Mesa", "centro", "rojo", "M", True, "todo año"],
35     "FL05": ["Ramo Bosque", "ramo", "verde", "L", False, "otoño"],
36     "FL06": ["Caja Noche", "caja", "morado", "M", True, "invierno"]
37 }
38
39 bodega = {
40     "FL01": [15990, 8],
41     "FL02": [29990, 3],
42     "FL03": [9990, 12],
43     "FL04": [24990, 5],
44     "FL05": [19990, 0],
45     "FL06": [22990, 6]
46 }
47
48
49 # =====
```

```
50 # 2. FUNCIONES DE VALIDACION
51 # =====
52 # Estas funciones reciben un dato, lo revisan y retornan:
53 # True -> si el dato es valido
54 # False -> si el dato no es valido
55 #
56 # Importante:
57 # Las funciones de validacion NO muestran mensajes.
58 # Los mensajes se muestran en las funciones que controlan el menu.
59 # =====
60
61 def validar_texto(texto):
62     """
63     Valida que un texto no este vacio ni tenga solo espacios.
64     """
65     return texto.strip() != ""
66
67
68 def validar_tamano(tamano):
69     """
70     Valida que el tamano sea S, M o L.
71     Se convierte a mayuscula para aceptar s, m o l.
72     """
73     tamano = tamano.strip().upper()
74     return tamano in ["S", "M", "L"]
75
76
77 def validar_tarjeta(respuesta):
78     """
79     Valida que la respuesta sea s o n.
80     """
81     respuesta = respuesta.strip().lower()
82     return respuesta in ["s", "n"]
83
84
85 def convertir_tarjeta(respuesta):
86     """
87     Convierte la respuesta del usuario a un valor booleano.
88     s -> True
89     n -> False
90     """
91     respuesta = respuesta.strip().lower()
92
93     if respuesta == "s":
94         return True
95     else:
96         return False
97
98
99 def validar_precio_positivo(precio):
100     """
101     Valida que el precio sea mayor que cero.
102     Se usa al agregar y actualizar precios.
103     """
104     return precio > 0
105
106
107 def validar_precio_no_negativo(precio):
108     """
109     Valida que el precio sea mayor o igual a cero.
110     Se usa para busquedas por rango de precio.
111     """
112     return precio >= 0
113
114
115 def validar_unidades(unidades):
116     """
117     Valida que las unidades sean mayores o iguales a cero.
118     """
119     return unidades >= 0
```

```
120
121
122 # =====
123 # 3. FUNCIONES PRINCIPALES DEL SISTEMA
124 # =====
125
126 def unidades_tipo(tipo):
127     """
128     Opcion 1:
129     Muestra el total de unidades disponibles segun el tipo de arreglo.
130
131     La busqueda no distingue entre mayusculas y minusculas.
132     Esta funcion no retorna valor; muestra el resultado por pantalla.
133     """
134     tipo = tipo.strip().lower()
135     total = 0
136
137     for codigo in arreglos:
138         datos_arreglo = arreglos[codigo]
139         tipo_arreglo = datos_arreglo[1].lower()
140
141         if tipo_arreglo == tipo:
142             unidades = bodega[codigo][1]
143             total = total + unidades
144
145     print("El total de unidades disponibles es:", total)
146
147
148 def busqueda_precio(p_min, p_max):
149     """
150     Opcion 2:
151     Busca arreglos cuyo precio este dentro del rango indicado.
152
153     Condiciones:
154     - Precio entre p_min y p_max.
155     - Unidades disponibles distintas de cero.
156     - Formato de salida: Nombre--Codigo.
157     - Resultados ordenados alfabeticamente por nombre.
158     """
159     encontrados = []
160
161     for codigo in bodega:
162         precio = bodega[codigo][0]
163         unidades = bodega[codigo][1]
164
165         if precio >= p_min and precio <= p_max and unidades != 0:
166             nombre = arreglos[codigo][0]
167             resultado = nombre + "--" + codigo
168             encontrados.append(resultado)
169
170     encontrados.sort()
171
172     if len(encontrados) == 0:
173         print("No hay arreglos en ese rango de precios.")
174     else:
175         print("Los arreglos encontrados son:", encontrados)
176
177
178 def actualizar_precio(codigo, nuevo_precio):
179     """
180     Opcion 3:
181     Actualiza el precio de un arreglo.
182
183     Retorna True si actualiza correctamente.
184     Retorna False si el codigo no existe.
185     """
186     codigo = codigo.strip().upper()
187
188     if codigo not in bodega:
189         return False
```

```
190
191     bodega[codigo][0] = nuevo_precio
192     return True
193
194
195 def agregar_arreglo(codigo, nombre, tipo, color_principal, tamano,
196                     incluye_tarjeta, temporada, precio, unidades):
197     """
198     Opcion 4:
199     Agrega un nuevo arreglo en ambos diccionarios.
200
201     Retorna True si agrega correctamente.
202     Retorna False si el codigo ya existe.
203     """
204     codigo = codigo.strip().upper()
205
206     if codigo in arreglos or codigo in bodega:
207         return False
208
209     arreglos[codigo] = [
210         nombre.strip(),
211         tipo.strip().lower(),
212         color_principal.strip().lower(),
213         tamano.strip().upper(),
214         incluye_tarjeta,
215         temporada.strip().lower()
216     ]
217
218     bodega[codigo] = [precio, unidades]
219     return True
220
221
222 def eliminar_arreglo(codigo):
223     """
224     Opcion 5:
225     Elimina un arreglo de ambos diccionarios.
226
227     Retorna True si elimina correctamente.
228     Retorna False si el codigo no existe.
229     """
230     codigo = codigo.strip().upper()
231
232     if codigo not in arreglos or codigo not in bodega:
233         return False
234
235     del arreglos[codigo]
236     del bodega[codigo]
237     return True
238
239
240 # =====
241 # 4. FUNCIONES DE APOYO PARA EL MENU
242 # =====
243
244 def mostrar_menu():
245     """
246     Muestra el menu principal del programa.
247     """
248     print()
249     print("===== MENU PRINCIPAL =====")
250     print("1. Unidades por tipo de arreglo")
251     print("2. Búsqueda de arreglos por rango de precio")
252     print("3. Actualizar precio de arreglo")
253     print("4. Agregar arreglo")
254     print("5. Eliminar arreglo")
255     print("6. Salir")
256     print("=====")
257
258
259 def pedir_respuesta_sn(mensaje):
```

```

260 """
261 Solicita una respuesta s/n.
262 Si el usuario ingresa otra cosa, vuelve a preguntar.
263 """
264 respuesta = input(mensaje).strip().lower()
265
266 while respuesta not in ["s", "n"]:
267     print("Debe ingresar s o n")
268     respuesta = input(mensaje).strip().lower()
269
270 return respuesta
271
272
273 # =====
274 # 5. CONTROL DE CADA OPCION DEL MENU
275 # =====
276
277 def opcion_unidades_tipo():
278     """
279     Controla la opcion 1 desde el programa principal.
280     """
281     tipo = input("Ingrese tipo de arreglo a consultar: ")
282
283     if not validar_texto(tipo):
284         print("El campo no puede estar vacio")
285     else:
286         unidades_tipo(tipo)
287
288
289 def opcion_busqueda_precio():
290     """
291     Controla la opcion 2.
292
293     Valida que:
294     - Los precios sean enteros.
295     - Los precios sean mayores o iguales a cero.
296     - El precio minimo no sea mayor que el maximo.
297     """
298     while True:
299         try:
300             p_min = int(input("Ingrese precio minimo: "))
301             p_max = int(input("Ingrese precio maximo: "))
302         except ValueError:
303             print("Debe ingresar valores enteros")
304             continue
305
306         if not validar_precio_no_negativo(p_min) or not validar_precio_no_negativo(p_max):
307             print("Los precios deben ser mayores o iguales a cero")
308             continue
309
310         if p_min > p_max:
311             print("El precio minimo no puede ser mayor que el precio maximo")
312             continue
313
314         break
315
316     busqueda_precio(p_min, p_max)
317
318
319 def opcion_actualizar_precio():
320     """
321     Controla la opcion 3.
322     Permite actualizar uno o mas precios.
323     """
324     continuar = "s"
325
326     while continuar == "s":
327         codigo = input("Ingrese codigo del arreglo: ")
328
329         if not validar_texto(codigo):

```

```
330     print("El codigo no puede estar vacio")
331 else:
332     try:
333         nuevo_precio = int(input("Ingrese nuevo precio: "))
334
335         if not validar_precio_positivo(nuevo_precio):
336             print("El precio debe ser mayor que cero")
337         else:
338             resultado = actualizar_precio(codigo, nuevo_precio)
339
340             if resultado:
341                 print("Precio actualizado")
342             else:
343                 print("El codigo no existe")
344
345     except ValueError:
346         print("Debe ingresar valores enteros")
347
348     continuar = pedir_respuesta_sn("'¿Desea actualizar otro precio (s/n)?': ")
349
350
351 def opcion_agregar_arreglo():
352     """
353     Controla la opcion 4.
354
355     Primero solicita todos los datos.
356     Luego valida cada dato.
357     Si una validacion falla, no registra el arreglo.
358     """
359     codigo = input("Ingrese codigo del arreglo: ")
360     nombre = input("Ingrese nombre: ")
361     tipo = input("Ingrese tipo: ")
362     color_principal = input("Ingrese color principal: ")
363     tamano = input("Ingrese tamano (S/M/L): ")
364     tarjeta = input("'¿Incluye tarjeta? (s/n)': ")
365     temporada = input("Ingrese temporada: ")
366
367     try:
368         precio = int(input("Ingrese precio: "))
369         unidades = int(input("Ingrese unidades: "))
370     except ValueError:
371         print("Debe ingresar valores enteros")
372         return
373
374     if not validar_texto(codigo):
375         print("El codigo no puede estar vacio")
376         return
377
378     if codigo.strip().upper() in arreglos or codigo.strip().upper() in bodega:
379         print("El codigo ya existe")
380         return
381
382     if not validar_texto(nombre):
383         print("El campo nombre no puede estar vacio")
384         return
385
386     if not validar_texto(tipo):
387         print("El campo tipo no puede estar vacio")
388         return
389
390     if not validar_texto(color_principal):
391         print("El campo color principal no puede estar vacio")
392         return
393
394     if not validar_texto(temporada):
395         print("El campo temporada no puede estar vacio")
396         return
397
398     if not validar_tamano(tamano):
399         print("El tamano debe ser S, M o L")
```

```
400         return
401
402     if not validar_tarjeta(tarjeta):
403         print("Debe ingresar s o n")
404         return
405
406     if not validar_precio_positivo(precio):
407         print("El precio debe ser mayor que cero")
408         return
409
410     if not validar_unidades(unidades):
411         print("Las unidades deben ser mayores o iguales a cero")
412         return
413
414     incluye_tarjeta = convertir_tarjeta(tarjeta)
415
416     resultado = agregar_arreglo(
417         codigo,
418         nombre,
419         tipo,
420         color_principal,
421         tamano,
422         incluye_tarjeta,
423         temporada,
424         precio,
425         unidades
426     )
427
428     if resultado:
429         print("Arreglo agregado")
430     else:
431         print("El codigo ya existe")
432
433
434 def opcion_eliminar_arreglo():
435     """
436     Controla la opcion 5.
437     """
438     codigo = input("Ingrese codigo del arreglo a eliminar: ")
439
440     if not validar_texto(codigo):
441         print("El codigo no puede estar vacio")
442         return
443
444     resultado = eliminar_arreglo(codigo)
445
446     if resultado:
447         print("Arreglo eliminado")
448     else:
449         print("El codigo no existe")
450
451
452 # =====
453 # 6. PROGRAMA PRINCIPAL
454 # =====
455
456 def main():
457     """
458     Controla el ciclo del menu.
459     El programa termina solo cuando el usuario selecciona la opcion 6.
460     """
461     opcion = ""
462
463     while opcion != "6":
464         mostrar_menu()
465         opcion = input("Ingrese opcion: ").strip()
466
467         if opcion == "1":
468             opcion_unidades_tipo()
469
```

```
470     elif opcion == "2":
471         opcion_busqueda_precio()
472
473     elif opcion == "3":
474         opcion_actualizar_precio()
475
476     elif opcion == "4":
477         opcion_agregar_arreglo()
478
479     elif opcion == "5":
480         opcion_eliminar_arreglo()
481
482     elif opcion == "6":
483         print("Programa finalizado.")
484
485     else:
486         print("Debe seleccionar una opcion valida")
487
488
489 # =====
490 # 7. INICIO DEL PROGRAMA
491 # =====
492
493 main()
```

13. Pruebas rápidas sugeridas

13.1. Prueba 1: buscar unidades por tipo

Entrada:

```
1 1
2 ramo
```

Resultado esperado:

```
1 El total de unidades disponibles es: 20
```

Explicación: los arreglos de tipo ramo son FL01, FL03 y FL05. Sus unidades son 8, 12 y 0. El total es 20.

13.2. Prueba 2: búsqueda por rango de precio

Entrada:

```
1 2
2 10000
3 25000
```

Resultado esperado:

```
1 Los arreglos encontrados son: ['Caja Noche--FL06', 'Centro Mesa--FL04', 'Ramo Primavera--FL01']
```

13.3. Prueba 3: precio incorrecto

Entrada:

```
1 2
2 hola
```

Resultado esperado:

```
1 Debe ingresar valores enteros
```

13.4. Prueba 4: agregar arreglo válido

Entrada:

```
1 4
2 FL010
3 Ramo Aurora
4 ramo
5 celeste
6 S
7 s
8 todo año
9 14990
10 7
```

Resultado esperado:

```
1 Arreglo agregado
```

13.5. Prueba 5: eliminar arreglo existente

Entrada:

```
1 5
2 FL010
```

Resultado esperado:

```
1 Arreglo eliminado
```

14. Errores frecuentes que deben evitarse

14.1. Copiar los puntos suspensivos

Incorrecto:

```
1 arreglos = {
2     "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"],
3     ...
4 }
```

Correcto:

```
1 arreglos = {
2     "FL01": ["Ramo Primavera", "ramo", "rosado", "M", True, "primavera"]
3 }
```

14.2. No convertir el código a mayúscula

Si el usuario escribe flo1, el programa debe encontrar FL01. Para lograrlo se usa:

```
1 codigo = codigo.strip().upper()
```

14.3. No validar números

Incorrecto:

```
1 precio = int(input("Ingrese precio: "))
```

Si el usuario escribe hola, el programa se cae.
Correcto:

```
1 try:
2     precio = int(input("Ingrese precio: "))
3 except ValueError:
4     print("Debe ingresar valores enteros")
```

15. Guion sugerido para desarrollar la clase

Tiempo	Actividad
10 min	Presentar el caso FlorExpress y explicar los dos diccionarios.
15 min	Construir el menú principal y explicar el ciclo while.
20 min	Resolver opción 1 y opción 2, enfatizando búsqueda y acumulación.
20 min	Resolver actualización de precio y explicar retorno True/False.
25 min	Resolver agregar arreglo con validaciones.
15 min	Resolver eliminación y salida del sistema.
15 min	Ejecutar pruebas rápidas y corregir errores frecuentes.

16. Cierre de la clase

Esta evaluación mide mucho más que escribir instrucciones en Python. También mide la capacidad de ordenar una solución, dividir el problema en funciones pequeñas, validar entradas y mantener un flujo de ejecución comprensible.

La recomendación para el estudiante es avanzar paso a paso:

1. Crear los diccionarios iniciales.
2. Crear el menú.
3. Implementar una opción a la vez.
4. Probar cada opción antes de continuar.
5. Agregar validaciones.
6. Revisar los mensajes de error.
7. Ejecutar una prueba completa desde el menú.

Mensaje final para estudiantes: programar bien no significa escribir todo rápido. Significa construir una solución ordenada, probarla por partes y corregir con calma. Paso a paso es que se llega lejos.